Notes for C++

Section 3 : Real numbers, Booleans, libraries, field widths and debugging

Real numbers always include a decimal fraction. Real numbers are used for measuring (e.g. weights, temperatures, etc) and for scientific calculations. Integers never have a decimal fraction and are used for counting. Here is a program that uses real numbers:

```
#include <iostream>
using namespace std;

float x, y, r;

int main() {
  x = 8.5;
  y = 1.33;
  r = x + y;
  cout << "The final answer is " << r;
}
```

Things to note about this program:

- we use `float` to define real variables
- real numbers are always written with a fraction, e.g. `8.5`, `1.33`, `2.0`, etc

Reading Real Numbers

```
#include <iostream>
using namespace std;

float x, y, r;

int main() {
  cout << "Enter the first number ";
  cin >> x;
  cout << "Enter the second number ";
  cin >> y;
  r = x + y;
  cout << "The final answer is " << r;
}
```

Operations with Real Numbers

The operations are similar to those available for integer numbers and include:

`x + y`	the add (or plus) operation
`x - y`	the subtract (or minus) operation
`x * y`	the multiply (or times) operation
`x / y`	the divide operation

Note that the divide operation produces the correct answer (including the fraction) for real numbers. For example: 1.0 / 2.0 produces 0.5 (whereas 1 / 2 produces 0 when using integer division).

At least one of the values involved must be a real number for fractions to appear in the result.

Note that there is ***no remainder operation*** for real numbers.

Mixed mode Arithmetic – some problems

Example 1:

We can use integer and real number variables in the same program. For example:

```
#include <iostream>
using namespace std;

int a;
float m;

int main() {
  a = 3;
  m = a;       /* m will be loaded with 3.0 */
  cout << "m is " << m;
  m = 6.8;
  a = m;       /* a will be loaded with 6 */
  cout << "a is " << a;
}
```

Notes:

- the computer always chops the fraction off a float when converting to an integer. This can be a problem if it isn't what you want to happen.
- but **truncation of a decimal value can be very useful** in a programming problem that requires you **to extract the whole number part out of a real number.**

Example 2:

The following program gives the wrong answer because of the integer division:

```
#include <iostream>
using namespace std;

int num1, num2;
float average;

int main() {
  num1 = 3;
  num2 = 4;
  average = (num1 + num2) / 2;
  cout << "The average is " << average;
}
```

We can fix it by ensuring that we use real values (not integers) in the division.

```
/* Fix, Version 1 - changing 2 to a real number */

#include <iostream>
using namespace std;
```

```
int num1, num2;
float average;

int main() {
  num1 = 3;
  num2 = 4;
  average = (num1 + num2) / 2.0;
  cout << "The average is " << average;
}
```

Note that the calculation is no longer integer division because 2.0 is a real number.

```
/* Fix, Version 2 - including an extra float variable */

#include <iostream>
using namespace std;

int num1, num2;
float temp, average;

int main() {
  num1 = 3;
  num2 = 4;
  temp = num1 + num2;
  average = temp / 2;
  cout << "The average is " << average;
}
```

Note that the calculation is no longer integer division because `temp` is a float variable.

The Boolean Data Type

Variables are always declared with a particular **data type**, for example: `int a, b, c;`

`int` tells the computer that the data type for these variables is **integer**.

Now we introduce the **Boolean data type** which can only be assigned `true` or `false`. For example:

```
#include <iostream>
using namespace std;

bool header;
int num1, num2;

int main() {
  cout << "Enter the first number ";
  cin >> num1;
  cout << "Enter the second number ";
  cin >> num2;
  header = false;
  if (num1 < 10) {
    if (header == false) {
      cout << "List of numbers less than 10\n";
      header = true;
```

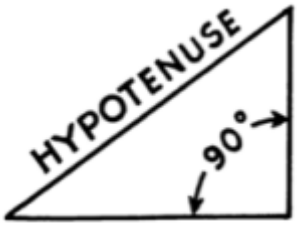
```
    }
    cout << num1 << endl;
  }
  if (num2 < 10) {
    if (header == false) {
      cout << "List of numbers less than 10\n";
      header = true;
    }
    cout << num2 << endl;
  }
  if (header == false) {
    cout << "There are no numbers less than 10\n";
  }
}
```

This program keeps track of whether the header has been displayed or not. If the header has not been displayed then it displays the other message about no numbers less than 10.

Boolean variables are used **to keep track of a situation**. For example, have I already displayed the heading?, is the current year a leap year?, is it raining today? We will discover other programs that use Boolean variables later on in the course. An if statement needs a condition. The condition can be a test or a Boolean variable. Note that you can never read or display Boolean variables.

Libraries and Functions

Calculating the length of the Hypotenuse



$$\text{Hypotenuse} = \sqrt{\text{side1}^2 + \text{side2}^2}$$

A right-angled triangle has side one, side two and the hypotenuse. The hypotenuse is the side opposite the right angle. Pythagoras' theorem (from 500BC) tells us that the square of the hypotenuse is equal to the sum of the squares of the other two sides. Thus:

$$\text{hypotenuse}^2 = \text{side1}^2 + \text{side2}^2$$

Then take the square root of both sides. Now we can write the program to calculate the hypotenuse:

```
#include <iostream>
#include <cmath>    // *IMPORTANT* - an extra #include

using namespace std;

float side1, side2, result, hypotenuse;

int main() {
  cout << "Enter the length of side one ";
  cin >> side1;
  cout << "Enter the length of side two ";
  cin >> side2;
```

```
  result = (side1 * side1) + (side2 * side2);
  hypotenuse = sqrt(result);
  cout << "The hypotenuse is " << hypotenuse << endl;
}
```

Functions and Libraries

Operations have symbols (e.g. `+, -, *, /`). ***Functions*** have a **name** (such as `sqrt`) and they often also have **parameters**. Zero or more parameters appear inside the brackets following the function name. Parameters are also sometimes called ***arguments***. Each function exists within a C++ ***library*** which is a file that contains a collection of functions. Each function can carry out one useful task. We have to `#include` the correct library name in order to use library functions in the program.

Calling a function

We say we **call** a function to use it. The format of a call to a function always involves the function's name followed by parentheses, which may or may not have something in them, depending on whether or not that function has parameters. In some cases the call is part of an assignment statement (see the call to `sqrt` above), but in others it is only the function name and the parentheses. Later we will learn how to write our own functions and the format will be fully explained. For now, we simply learn the format of a call for each library function we use.

The `cmath` library

One of the functions used in the example program above is called `sqrt` and we have to `#include` the `cmath` library to use this function. The `sqrt` function takes one parameter in the brackets. In the above example, the value in the variable called result is the parameter. The `cmath` library contains many other functions that you can use in your programs, such as `sin, cos, fabs, pow` and many more. Here is the full list of available functions in the `cmath` library:
http://www.cplusplus.com/reference/cmath/

The `iostream` library

We have to `#include` the `iostream` library to be able to use input/output functions. `cout` and `cin` are objects in the `iostream` library. More about objects later on.

The `iomanip` library

Provides functions that manipulate the display. For example, see field widths in this section.
http://www.cplusplus.com/reference/iomanip/

The `cstdlib` library

This is a useful library with many interesting functions in it:
http://www.cplusplus.com/reference/cstdlib

Some example functions are:

exit(); *you need to know how to use this function*

`exit()` is used to stop a program. The program will end when the `exit()` function is executed. Put any integer number inside the brackets. Here is an example program:

```
#include <iostream>
#include <cstdlib>

using namespace std;

int value, result;

int main() {
  cout << "Enter a value greater than 10 ";
  cin >> value;
  if (value < 10) {
     cout << "ERROR: the value is not greater than 10";
     exit(13);
  }
  result = value + 4;
  cout << "The result is " << result;
}
```

abs(); *you need to know how to use this function*

`abs()` is used to calculate the absolute value of an integer value. It has one integer parameter. Note that there are other functions (in other libraries) to calculate the absolute value of real numbers (or other data types). In particular `fabs()` in the `cmath` library is used to calculate absolute values for real numbers.

Here is an example program to calculate the age difference between two people:

```
#include <iostream>
#include <cstdlib>

using namespace std;

int age1, age2, difference;

int main() {
  age1 = 25;
  age2 = 27;
  difference = abs(age1 - age2);
  cout << "The age difference is " << difference << " years.";
}
```

rand() and **srand();** *these functions are *not* needed in our course but are interesting*

`rand()` produces a random number. Here is an example program to simulate throwing one dice three times. We also need `srand()` to create a different set of random numbers each time.

```
#include <iostream>
#include <cstdlib>
```

```
using namespace std;

int value, seed;

int main() {
  cout << "Enter the random number seed ";
  cin >> seed;
  srand(seed);
  cout << "Throw a dice three times...\n";
  value = rand() % 6 + 1;
  cout << "The first throw is " << value << endl;
  value = rand() % 6 + 1;
  cout << "The second throw is " << value << endl;
  value = rand() % 6 + 1;
  cout << "The third throw is " << value << endl;
}
```

If you do not have seed (or you enter the same seed each time) the program will give exactly the same sequence of dice rolls. It only becomes random if you enter a different seed each time.

Using namespace

In most programming languages, including C++, every name has to be unique. Thus every function and every variable must have a different name. Sometimes, it can be a problem thinking up unique names. C++ has a system of placing objects within containers (known as namespaces) and this means that names (variables, functions, etc) exist within that namespace. They could then have the same name as another object but this is not a problem as long as the two objects are inside different namespaces.

C++ moved many standard objects and operations into the ***standard namespace*** (called `std` for short). For example, the full name of `cout` is `std::cout` which means that `cout` belongs to the `std` namespace. Similarly, the full name of `cin` is `std::cin`.

We can either use the full names or we can put `using namespace std` at the top of our program which means that `std::` is automatically added to the front of any objects we may be using.

As an experiment, leave out the line `using namespace std` and see what happens.

Using Field Widths with Integer output

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is " << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

The display will look like this:

```
The value is 37
The value is 1005
```

Sometimes, we want to line numbers up on the right, like this:

```
The value is   37
The value is 1005
```

Note the **width of the display** (i.e. the number of places on the screen). Each line in this display has a width of 17. Each value in this display has a width of 4. Count the characters on the display to make sure. We can use `cout.width` and `right` (or `left`) to arrange the display as follows:

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is ";
  cout.width(4);              // width of the display will be 4
  cout << right << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

Sometimes, we want to show **leading zeros**, like this:

```
The value is 0037
The value is 1005
```

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is ";
  cout.fill('0');          // fill extra places with zeros
  cout.width(4);           // width of the display will be 4
  cout << right << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

Note that if a width is too small, the computer ignores it, as in this example:

```
#include <iostream>
using namespace std;

int value;
```

```
int main() {
  value = 8462;
  cout << "The value is ";
  cout.width(2);              // width of the display will be 2
  cout << right << value << endl;
}
```

Using Field Widths with Real number output

The above discussion of width, left, right and fill can also be used with real numbers. In addition, real numbers can make use of `setprecision` to set the number of decimal places to be displayed. Along with `setprecision` we use `fixed` to fix the number of decimal places to be the same for all numbers in the display. Note that we need the `iomanip` library to use `setprecision`.

```
#include <iostream>
#include <iomanip>
using namespace std;

float result;

int main() {
  result = 1.0 / 3.0;   // display this result to 7 decimal places
  cout << "The result is ";
  cout << setprecision(7) << fixed << result << endl;
}
```

Warning – you may not be displaying the correct result

It is possible to display incorrect results. This could be potentially dangerous as an observer may not be aware that incorrect results are being displayed. Here is an example of an incorrect display:

```
#include <iostream>
#include <iomanip>
using namespace std;

float result;

int main() {
  result = 2.0 / 3.0;   // display this result to 1 decimal place
  cout << "The result is ";
  cout << setprecision(1) << fixed << result << endl;
}
```

Example : A program to calculate GST

```
#include <iostream>
#include <iomanip>
using namespace std;

/* this program calculates GST */

float price, gst, total, rate;
```

```
int main() {
  cout << "Enter the price of the item $";
  cin >> price;
  rate = 15.0 / 100.0;     // check that GST is 15.0%
  gst = price * rate;
  /* fix all numbers to display 2 decimal places */
  cout << setprecision(2) << fixed;
  cout << "GST on this item is $" << gst << endl;
  total = price + gst;
  cout << "Total to pay is $" << total << endl;
}
```

Another Example : A program to convert metres to the imperial system

Lengths are measured in New Zealand using the ***metric system***. For example, a basketball court in New Zealand is 28.0 metres long. However, in the USA, distances and lengths are measured using the ***imperial system*** which uses yards, feet and inches. In the USA the length of a basketball court is 31 yards, 1 foot, 0 inches. Are basketball courts actually the same length in the two countries?

Write a program that reads in a length in metres and converts the length to yards, feet and inches. The metres and the inches must contain fractions (real numbers) but the yards and feet do not contain fractions (integer numbers). The output from your program should look like this:

Example 1:

```
Enter a length in metres: 3.497
3.497 metres = 3 yards 2 feet 5.7 inches
```

Example 2:

```
Enter a length in metres: 1.245
1.245 metres = 1 yards 1 feet 1.0 inches
```

(should it be foot or feet..? don't worry about this!)

Note: 1 metre = 39.3701 inches; 12 inches = 1 foot (feet); 3 feet = 1 yard

```
#include <iostream>
#include <iomanip>
using namespace std;

float metres, inches;
int yards, feet;

int main() {
  cout << "This program converts metres into the imperial system\n";
  cout << "Enter a length in metres: ";
  cin >> metres;
  inches = metres * 39.3701;
  feet = inches / 12;          // real division loaded into an int
  yards = feet / 3;
  feet = feet % 3;
  inches = inches - yards * 36 - feet * 12;
```

```
  cout << setprecision(3) << fixed;
  cout << metres << " metres = ";
  cout << yards << " yards " << feet << " feet ";
  cout << setprecision(1) << fixed;
  cout << inches << " inches " << endl;
}
```

Debugging

A problem in a program is called a bug and the process of finding the bugs in a program is called debugging. Programmers spend an enormous amount of time debugging. It is really worthwhile to develop some good habits with your programming style to reduce the amount of time spent debugging.

Software tools to assist with debugging

People have written programs called debuggers which are designed to assist programmers in finding bugs. Generally, debuggers keep track of which instruction the program is currently executing and the current values of all the variables. The use of debuggers in a first-year course is not recommended because they are usually quite difficult to understand and use.

You can read more about debuggers here: (this is not part of our course)
https://en.wikipedia.org/wiki/Debugger

Types of Error

Good programmers know the different types of errors that may occur in a program.

Syntax errors

Errors in the syntax, i.e. the statement is written incorrectly. An example of a syntax error is:

```
  cout << "Hello world
```

The compiler picks up syntax errors, tells you about them, and refuses to run the program. Get used to the syntax errors that are reported by the system (it usually shows you the line number which is useful, and describes what the error is). Make use of these error messages to fix any syntax errors that may occur. Syntax errors are the easiest kind of errors to fix because the compiler tells you about them.

Logic errors

The compiler does not report logic errors and the program runs but produces the wrong results. Logic errors can be very difficult to find because there is nothing wrong with the syntax of the program. For example, you may write a program to add two numbers and the program may run perfectly. But if you were meant to write a program ***multiplies*** two numbers, then you have a program that runs correctly but it is not doing what is required.

The **really difficult logic errors** are when a program works correctly some of the time but not for every different situation that may arise.

In our course we will use these simple definitions of errors:
Syntax error – an error where an error message is reported by the compiler.
Logic error – no error message appears but the program does not do what is required.

There are other types of errors, but we will not worry about them. For example, semantic errors, run-time errors and so on. In fact, many programmers don't agree on the definitions for certain errors!

Make your programs easy to read

Some programmers love to make their programs **easy to write**. Using shortcuts, having short variable names (e.g. a and b) and other tricks make it faster to type in your code. We are not interested in entering code quickly because we know that we don't spend much time on entering code anyway. We spend a huge amount of time going slowly through our programs looking for errors. So, it is a much better idea to make your programs **easy to read**.

Here are some of the ways to make your program easier to read:

- Use meaningful variable names
- Use indentation correctly (especially for nested statements)
- Use appropriate spacing in calculations
- Place a few comments in your program explaining what is being calculated

None of these things are **rules** in the C++ language. But you need to get into good programming **habits** and have good **programming style**. It will help enormously later on when you are trying to debug programs that are much bigger and a lot more complicated.

Display variables as you go

Programmers like to use debuggers to assist with debugging because they keep track of the values in each variable. However, you can always keep track of any variable by including extra cout statements in your program. This is especially important for variables that are not usually displayed as part of the results.

For example, in the following program I use the variable temp. This is an important variable but it is not displayed with the final results. I add some debugging cout statements to the program to display temp.

```
#include <iostream>
using namespace std;

int price, cash, temp;

int main() {
  cout << "NOTE: enter all amounts as cents.\n";
  cout << "For example, $8.97 is entered as 897\n";
  cout << "Enter the price on the item: ";
  cin >> price;
  temp = price + 5;
  cout << "DEBUG: temp after price + 5 is " << temp << endl;
  temp = temp / 10;
  cout << "DEBUG: temp after temp / 10 is " << temp << endl;
  cash = temp * 10;
  cout << "The price is " << price;
  cout << " and you must pay " << cash;
}
```

When the program is complete and after all testing, we can remove those DEBUG cout statements.

Use comments to remove some code in your programs

When there are many lines of code, it is much easier to check only a small part of the program at a time for errors. We can use comments to temporarily remove some instructions. By doing this we can test only the remaining instructions. When everything is working correctly, we can take out the comments and once more include the commented code in the program.

This program has an error in it. Can you see the error? We use comments to remove code temporarily to locate a problem and we include a debugging cout statement.

```
/* A program to find the biggest of three values */
#include <iostream>
using namespace std;

int a, b, c, big;

int main() {
  cout << "Enter three integer values\n";
  cin >> a >> b >> c;
  big = a;
  if (big > b) {
    big = b;
  }
  cout << "DEBUG: The biggest number is " << big;
}
/*if (big < c) {
    big = c;
  }
  cout << "The biggest number is " << big; */
}
```

The debug cout helps to see what is wrong and the commented code will not run. This allows you to focus on the fact that the first `if` statement isn't working properly and then to find the error in that `if`. For long and complicated programs this use of comments is invaluable.

Testing

Testing is extremely important and is the only way to find logic errors. You must always test your programs and you need to test for every possible type of situation. Here are some general notes about testing:

- Set up examples during the programming process (see the lecture on the programming process)
- Set up at least one example for each possible situation
- Make the data in your examples as simple as possible
- Calculate the results yourself before you run the program (know what the result should be)
- Run your program and test every example – check that the results match your examples
- Ask a friend to try out your program